

ECE 473/573
Cloud Computing and Cloud Native Systems
Lecture 09 SQL Database I

Professor Jia Wang
School of Engineering, Applied Science, and Health
Illinois Institute of Technology

September 16, 2026

Outline

Persistence and Cloud Storage

Relational Data Model and SQL

Accessing SQL from Go

Reading Assignment

- ▶ This and next lecture: SQL Database
 - ▶ Project 2 documentation and source code.

Outline

Persistence and Cloud Storage

Relational Data Model and SQL

Accessing SQL from Go

Persisting Marketplace State

- ▶ In practice, instead of maintaining its own transaction log, most RESTful services delegate persistence to a database service.
- ▶ Project 2
 - ▶ Marketplace service → PostgreSQL → Persistent volume
 - ▶ The Go service handles RESTful requests and business logic.
 - ▶ PostgreSQL stores accounts, listings, and orders.
 - ▶ Docker manages the persistent volume.
- ▶ How storage requirements are handled on cloud in general?

Cloud Storage

- ▶ A fundamental component of cloud computing.
- ▶ Block storage and file systems
 - ▶ Provide byte blocks of fixed size, usually managed as file systems consisting of files and directories.
 - ▶ E.g. storage for PostgreSQL's database files.
- ▶ Database systems
 - ▶ Provide accesses to highly structured data beyond read/write.
 - ▶ E.g. find available listings or transfer credits for a purchase.
- ▶ Object storage
 - ▶ Manage data as objects that must be modified as a whole, which enables many performance optimizations.
 - ▶ Use a key as identifier instead of a name.
 - ▶ E.g. product images that rarely change – a marketplace would store them outside of the database.

Outline

Persistence and Cloud Storage

Relational Data Model and SQL

Accessing SQL from Go

Relational Database

- ▶ A classical approach for data management.
 - ▶ Use relational algebra as a foundation for querying tables, usually captured by the SQL language.
 - ▶ Provide ACID guarantee on database operations including data persistency and concurrent access.
- ▶ Usually run as a stand-alone service that clients can access locally or remotely.
 - ▶ Via management tools, or
 - ▶ Via APIs that are available from most programming languages.

Data Models in Relational Database

- ▶ Data are organized into tables or relations.
- ▶ Each table consists of many rows or tuples of data.
 - ▶ A row (or tuple) describes one record.
- ▶ Each row consists of many columns or attributes or fields
 - ▶ Rows in the same table should have the same columns.
- ▶ Each column of a row is usually of an elementary data type.
 - ▶ That can be compared and operated on.
 - ▶ Opaque binary blobs are also supported by many database systems to store data like images.

Marketplace Tables

Table	Selected columns
accounts	account_id, balance
listings	listing_id, seller_id, credit_cost, status
orders	order_id, listing_id, buyer_id, seller_id credit_cost, status

- ▶ One account may act as both a buyer and a seller.
- ▶ A listing belongs to one seller.
- ▶ An order refers to a listing, a seller, and a buyer.

Example: Buying a Used Textbook

accounts (before → after)

account_id	balance
Alice	100 → 125
Bob	100 → 75

listings (after)

listing_id	seller_id	credit_cost	status
listing-1	Alice	25	sold

orders (after)

order_id	listing_id	buyer_id	seller_id	credit_cost	status
order-1	listing-1	Bob	Alice	25	completed

- ▶ Initially Alice and Bob each have 100 credits.
- ▶ Alice lists a used textbook for 25 credits; Bob buys it.
- ▶ An order is Bob's purchase and Alice's sale.

Keys and Relationships

- ▶ A primary key (or key) uniquely identifies a row in its table.
 - ▶ Allow one to quickly locate the row given its key.
 - ▶ Additionally to support a range query of keys.
 - ▶ E.g. `account_id`, `listing_id`, and `order_id`.
 - ▶ A primary key may consist of more than one column.
- ▶ A foreign key refers to a key in another table.
 - ▶ E.g. `listings.seller_id` refers to `accounts.account_id`, and `orders.listing_id` refers to `listings.listing_id`.
- ▶ Foreign keys relate multiple tables
 - ▶ For complex queries.
 - ▶ For data validation during updates.
 - ▶ E.g. prevent a listing from referring to to nonexistent account.

Creating Tables

```
-- From database/schema.sql in Project 2
CREATE TABLE accounts (
    account_id TEXT PRIMARY KEY,
    balance INTEGER NOT NULL CHECK (balance >= 0)
);
```

- ▶ SQL: Structured Query Language
 - ▶ A language describing which data to retrieve or change.
- ▶ **CREATE TABLE** creates a table from a list of columns.
 - ▶ **NOT NULL** requires a value; **CHECK** restricts allowed values.
 - ▶ The database enforces these rules for every client.

Selecting and Filtering

```
SELECT balance
FROM accounts
WHERE account_id = 'Bob';
```

```
SELECT listing_id, title, credit_cost
FROM listings
WHERE status = 'available';
```

- ▶ **SELECT** queries the database and returns data as rows
 - ▶ Columns are specified in the **SELECT** clause.
 - ▶ E.g. one column **balance** is generated for the first query.
- ▶ **FROM** specifies the table(s) those data come from.
- ▶ **WHERE** selects rows satisfying a condition.

Seller Listings and Output Ordering

```
SELECT listing_id, seller_id, title, credit_cost, status, listed_at
FROM listings
WHERE seller_id = 'Alice'
ORDER BY listed_at, listing_id;
```

- ▶ Retrieve all listings from Alice, including sold listings.
- ▶ Use **ORDER BY** to sort the output rows.
 - ▶ Either ascending (**ASC**, default) or descending (**DESC**).
 - ▶ listing_id breaks ties when listed_at values are equal.

Join

```
SELECT o.order_id, o.title, o.credit_cost, a.account_id, a.balance
FROM orders AS o JOIN accounts AS a ON o.buyer_id = a.account_id
WHERE a.balance >= 100;
```

- ▶ Can we locate accounts with enough balance and look into their past orders to recommend something for them to buy?
 - ▶ You may write separate queries but it is convenient to allow the database to run and optimize them.
- ▶ **JOIN** combines matching rows from multiple tables.
 - ▶ Match orders.buyer_id with accounts.account_id.
- ▶ Aliases **o** and **a** provide short names for the tables.

Grouping and Aggregation

```
SELECT buyer_id, SUM(credit_cost) AS total_spending
FROM orders
WHERE status = 'completed'
GROUP BY buyer_id
ORDER BY total_spending DESC, buyer_id;
```

- ▶ Can we also find accounts spending the most?
- ▶ Use **GROUP BY** to form one group per buyer.
- ▶ Rows from each group are aggregated into a new row.
 - ▶ Via aggregate functions like **SUM** to calculate the total spending of each buyer.
- ▶ **ORDER BY** ranks buyers by spending, breaking ties with `buyer_id`.

Changing Data

```
INSERT INTO listings (seller_id, title, credit_cost)
VALUES ('Alice', 'Used textbook', 25)
RETURNING listing_id, status, listed_at;
```

```
UPDATE listings
SET status = 'sold'
WHERE listing_id = 'listing-1';
```

- ▶ **INSERT** creates rows; **UPDATE** modifies rows.
- ▶ **RETURNING** retrieves columns from the inserted or updated rows, usually for those filled by the database automatically.

Indexes and Access Patterns

```
SELECT o.order_id, o.title, o.credit_cost, a.account_id, a.balance
FROM orders AS o JOIN accounts AS a ON o.buyer_id = a.account_id
WHERE a.balance >= 100;
```

```
SELECT buyer_id, SUM(credit_cost) AS total_spending
FROM orders
WHERE status = 'completed'
GROUP BY buyer_id
ORDER BY total_spending DESC, buyer_id;
```

```
CREATE INDEX orders_buyer_id_index
ON orders (buyer_id, ordered_at, order_id);
```

- ▶ The orders table has a primary key of order_id.
 - ▶ Query based on order_id can be optimized.
- ▶ An index optimizes access to rows using different columns.
 - ▶ This index matches buyer filtering.
- ▶ Indexes have costs when inserting and updating data.
 - ▶ Choose indexes based on the queries the application needs.

Outline

Persistence and Cloud Storage

Relational Data Model and SQL

Accessing SQL from Go

Project 2: from RESTful Requests to SQL

HTTP request → REST Handler → Store → SQL query

- ▶ Responsibilities are divided into multiple source files.
 - ▶ `main.go`: setup REST handlers.
 - ▶ `handlers.go`: decode requests and encode responses.
 - ▶ `marketplace` package: business logic and database operations.
- ▶ A database row need not match a complete API response.
 - ▶ The `Account` object returned by a RESTful request should include balance, listings, orders, and sales.
 - ▶ These fields are assembled from several queries.

Reading One Row

```
var balance int
err := s.db.QueryRow(
    "SELECT balance FROM accounts WHERE account_id = $1",
    accountID,
).Scan(&balance)
// omitting error handling
return balance, nil
```

- ▶ Avoid the SQL injection attack
 - ▶ DO NOT format the query by string operations.
 - ▶ Instead, pass values separately via a placeholder like `$1`.
- ▶ Scan copies the result columns into Go variables.
- ▶ If multiple rows match, `QueryRow` only returns the first.

Reading Multiple Rows

```
rows, err := s.db.Query(`
    SELECT listing_id, seller_id, title, credit_cost, status, listed_at
    FROM listings
    WHERE seller_id = $1
    ORDER BY listed_at, listing_id`,
    accountID,
)
// omitting error handling
defer rows.Close()
...
```

- ▶ Avoid the SQL injection attack
 - ▶ DO NOT format the query by string operations.
 - ▶ Instead, pass values separately via a placeholder like `$1`.
- ▶ Query returns a set of rows to iterate over.
 - ▶ Use a loop to scan each row and append to a Go slice.
 - ▶ `defer rows.Close()` releases associated resources.

Summary

- ▶ Use a database system to persist states.
- ▶ Check <https://www.w3schools.com/sql/default.asp> and run examples there to learn SQL.
- ▶ Check <https://go.dev/doc/database/querying> to learn Go database queries.